```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
import kagglehub
andrewmvd_hard_hat_detection_path = kagglehub.dataset_download('andrewmvd/hard-hat-detection')

print('Data source import complete.')
```

```
import torch
import torchvision
import os
from pathlib import Path
import shutil
from PIL import Image
import json
import xml.etree.ElementTree as ET
from IPython import display
from tqdm.notebook import tqdm
import random
import yaml
import ultralytics
from ultralytics import YOLO
import pandas as pd
```

```
print(torch.__version__)
print(torchvision.__version__)
```

```
2.6.0+cu124
0.21.0+cu124
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16
else:
    device = "cpu"
    dtype = torch.float16

print(f" Device = {device} , dtype = {dtype}")
```

```
 Device = cuda , dtype = torch.float16
```

```
# downlaod the dataset
import kagglehub

# Download latest version
path = kagglehub.dataset_download("andrewmvd/hard-hat-detection")

print("Path to dataset files:", path)
```

```
Path to dataset files: /kaggle/input/hard-hat-detection
```

```
# move  the images and the annotations
image_dir = Path("/kaggle/working/data/images")
anno_dir = Path("/kaggle/working/data/annotations")
image_dir.mkdir(parents=True ,exist_ok=True)
anno_dir.mkdir(parents=True ,exist_ok=True)
if image_dir.is_dir() and anno_dir.is_dir():
    print(f"{image_dir} and {anno_dir} are created.")
```

```
/kaggle/working/data/images and /kaggle/working/data/annotations are created.
```

```
# Define source and destination paths
src_path = Path("/kaggle/input/hard-hat-detection/images")
dest_dir = Path("/kaggle/working/data/images")

# Create destination directory if it doesn't exist
```

```
dest_dir.mkdir(parents=True , exist_ok=True)

# Loop through files and copy
for file in src_path.iterdir():
    if file.is_file():  # Optional: Ensures it's a file
        file_name = file.name
        src_file = src_path / file_name
        dest_file = dest_dir / file_name
        shutil.copy(src_file, dest_file)
print(f"File copied from {src_path}  ,to {dest_dir}")
```

```
File copied from /kaggle/input/hard-hat-detection/images  ,to /kaggle/working/data/images
```

```
# move the annotaions now
# Define source and destination paths
src_path = Path("/kaggle/input/hard-hat-detection/annotations")
dest_dir = Path("/kaggle/working/data/annotations")

# Create destination directory if it doesn't exist
dest_dir.mkdir(parents=True , exist_ok=True)

# Loop through files and copy
for file in src_path.iterdir():
    if file.is_file():  # Optional: Ensures it's a file
        file_name = file.name
        src_file = src_path / file_name
        dest_file = dest_dir / file_name
        shutil.copy(src_file, dest_file)
print(f"File copied from {src_path}  ,to {dest_dir}")
```

```
File copied from /kaggle/input/hard-hat-detection/annotations  ,to /kaggle/working/data/annotations
```

```
images_list = [item.name for item in image_dir.iterdir()]
annotations_list = [item.name for item in anno_dir.iterdir()]
```

```
print(len(images_list))
print(len(annotations_list))
```

```
5000
5000
```

```
images_list[:10]
```

```
['hard_hat_workers164.png',
 'hard_hat_workers3036.png',
 'hard_hat_workers2298.png',
 'hard_hat_workers3707.png',
 'hard_hat_workers1278.png',
 'hard_hat_workers4136.png',
 'hard_hat_workers3235.png',
 'hard_hat_workers2642.png',
 'hard_hat_workers2088.png',
 'hard_hat_workers4614.png']
```

```
annotations_list[:10]
```

```
['hard_hat_workers2520.xml',
 'hard_hat_workers661.xml',
 'hard_hat_workers2170.xml',
 'hard_hat_workers2069.xml',
 'hard_hat_workers759.xml',
 'hard_hat_workers1752.xml',
 'hard_hat_workers2138.xml',
 'hard_hat_workers487.xml',
 'hard_hat_workers773.xml',
 'hard_hat_workers191.xml']
```

```
image_path = Path("/kaggle/working/data/images") / "hard_hat_workers1867.png"
Image.open(image_path)
```


```
# now we see the annotations files ok
training_dir = Path("/kaggle/working/data_images")
images_dir = training_dir / "images"
annotations_dir = training_dir / "annotations"
print("Images     :", images_dir)
print("Annotations:", annotations_dir)
```

```
Images     : /kaggle/working/data_images/images
Annotations: /kaggle/working/data_images/annotations
```

```
images_dir.mkdir(parents=True ,exist_ok=True)
annotations_dir.mkdir(parents=True , exist_ok=True)
if images_dir.is_dir() and annotations_dir.is_dir():
    print("both folder created.")
```

```
both folder created.
```

```
# move the annotaions now
# Define source and destination paths
src_path = Path("/kaggle/working/data/images")
dest_dir = Path("/kaggle/working/data_images/images")

# Create destination directory if it doesn't exist
dest_dir.mkdir(parents=True , exist_ok=True)

# Loop through files and copy
for file in src_path.iterdir():
    if file.is_file():  # Optional: Ensures it's a file
        file_name = file.name
        src_file = src_path / file_name
        dest_file = dest_dir / file_name
        shutil.move(src_file, dest_file)
print(f"File moved from {src_path}  ,to {dest_dir}")
```

```
File moved from /kaggle/working/data/images  ,to /kaggle/working/data_images/images
```

```
# move the annotaions now
# Define source and destination paths
src_path = Path("/kaggle/working/data/annotations")
dest_dir = Path("/kaggle/working/data_images/annotations")

# Create destination directory if it doesn't exist
dest_dir.mkdir(parents=True , exist_ok=True)

# Loop through files and copy
for file in src_path.iterdir():
    if file.is_file():  # Optional: Ensures it's a file
        file_name = file.name
        src_file = src_path / file_name
        dest_file = dest_dir / file_name
```

```
        shutil.move(src_file, dest_file)
print(f"File moved from {src_path}  ,to {dest_dir}")
```

```
File moved from /kaggle/working/data/annotations  ,to /kaggle/working/data_images/annotations
```

```
images_list = [item.name for item in images_dir.iterdir()]
annotations_list = [item.name for item in annotations_dir.iterdir()]
```

```
print(len(images_list))
print(len(annotations_list))
```

```
5000
5000
```

```
images_list[:10]
```

```
['hard_hat_workers164.png',
 'hard_hat_workers3036.png',
 'hard_hat_workers2298.png',
 'hard_hat_workers3707.png',
 'hard_hat_workers1278.png',
 'hard_hat_workers4136.png',
 'hard_hat_workers3235.png',
 'hard_hat_workers2642.png',
 'hard_hat_workers2088.png',
 'hard_hat_workers4614.png']
```

```
image_path = images_dir  / "hard_hat_workers3179.png"
Image.open(image_path)
```

```
annotation_path = annotations_dir / "hard_hat_workers4474.xml"
```

```
!head -n 25 $annotation_path
```

```
<annotation>
    <folder>images</folder>
    <filename>hard_hat_workers4474.png</filename>
    <size>
        <width>416</width>
        <height>416</height>
        <depth>3</depth>
    </size>
    <segmented>0</segmented>
    <object>
        <name>helmet</name>
        <pose>Unspecified</pose>
        <truncated>0</truncated>
        <occluded>0</occluded>
        <difficult>0</difficult>
        <bndbox>
            <xmin>130</xmin>
```

```
            <ymin>142</ymin>
            <xmax>155</xmax>
            <ymax>164</ymax>
        </bndbox>
    </object>
    <object>
        <name>helmet</name>
```

```
#  we have these classes in the dataset but not confirm yet
classes = ["helmet" , "head" , "person"]
```

```
class_mapping = {cls:idx for idx,cls in enumerate(classes)}
print(class_mapping)
```

```
{'helmet': 0, 'head': 1, 'person': 2}
```

```
# here we just go for practise and  understand  how we give the bouniding bxes to the yolo
width = 1200
height = 800
xmin = 833
ymin = 390
xmax = 1087
ymax = 800
```

```
# we compute the center of bounding box and then normalized it
x_center = (xmax + xmin) / 2 / width
y_center =(ymax + ymin) / 2 / height
print(f"X_center :{x_center} , Y_center : {y_center}")
```

```
X_center :0.8 , Y_center : 0.74375
```

```
# now we compute the length of the bounidng box in the images
bb_width = (xmax - xmin) / width
bb_height = (ymax - ymin) / height
print(f"BB_Width : {bb_width:0.3f} , BB_height : {bb_height:0.3f}")
```

```
BB_Width : 0.212 , BB_height : 0.512
```

```
def xml_to_yolo_bbox(bbox, width, height):
    """Convert the XML bounding box coordinates into YOLO format.

    Input:  bbox    The bounding box, defined as [xmin, ymin, xmax, ymax],
                    measured in pixels.
            width   The image width in pixels.
            height  The image height in pixels.

    Output: [x_center, y_center, bb_width, bb_height], where the bounding
            box is centered at (x_center, y_center) and is of size
            bb_width x bb_height.  All values are measured as a fraction
            of the image size."""

    xmin, ymin, xmax, ymax = bbox
    x_center = (xmin + xmax) / 2 / width
    y_center = (ymin + ymax) / 2 / height
    bb_width = (xmax - xmin) / width
    bb_height = (ymax - ymin )/ height

    return [x_center, y_center, bb_width, bb_height]


xml_to_yolo_bbox([xmin, ymin, xmax, ymax], width, height)
```

```
[0.8, 0.74375, 0.21166666666666667, 0.5125]
```

```
# now we are going to create the function of parse annotations that will read the bboxes from the xml
def parse_annotations(f):
    objects = []
    tree   = ET.parse(f)
    root   = tree.getroot()
    width  = int(root.find("size").find("width").text)
    height = int(root.find("size").find("height").text)
    for obj in root.findall("object"):
```

```
            label = obj.find("name").text
            class_id = class_mapping[label]
            bbox = obj.find("bndbox")
            xmin = int(bbox.find("xmin").text)
            xmax = int(bbox.find("xmax").text)
            ymin = int(bbox.find("ymin").text)
            ymax = int(bbox.find("ymax").text)
            yolo_bbox = xml_to_yolo_bbox([xmin,ymin,xmax,ymax] , width ,height)
            objects.append([class_id] + yolo_bbox )
        return objects
```

```
objects = parse_annotations(annotation_path)
```

```
objects[0]  # all are the helmet objects so go on son be a good one
```

```
[0,
 0.3425480769230769,
 0.36778846153846156,
 0.06009615384615385,
 0.052884615384615384]
```

```
# write a lable function . and i hope you get that what is label function is
# making the bounding box of object i a text format , help in training
def write_label(objects , filename):
    with open(filename , "w") as f:
        for obj in objects:
            f.write(" ".join(str(x) for x in obj))
            f.write("\n")
```

```
write_label(objects, "yolo_test.txt")
!head -n 2 yolo_test.txt
```

```
0 0.3425480769230769 0.36778846153846156 0.06009615384615385 0.052884615384615384
0 0.4639423076923077 0.46033653846153844 0.038461538461538464 0.040865384615384616
```

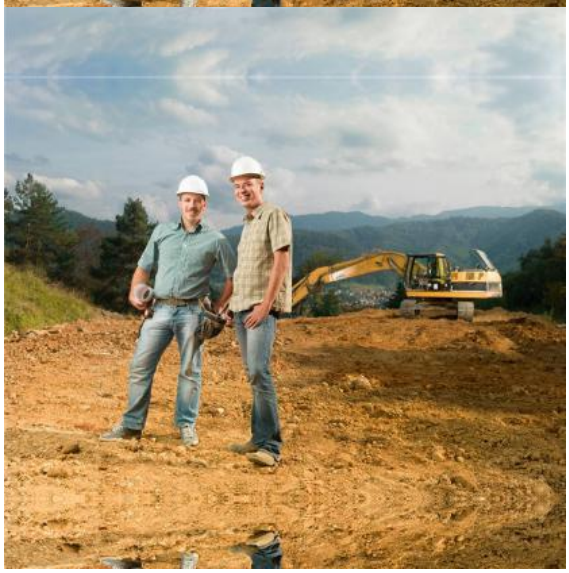
```
# we check that what type of images our dataset is contain
extentions=set(list([ x.suffix for x in images_dir.glob("*")]))
extentions # all are .png
```

```
{'.png'}
```

```
# for yolo it is best practise to convert all of the inot jped formatok
def image_conversion(fin ,fout):
    Image.open(fin).convert("RGB").save(fout , "JPEG")
```

```
test_image = images_dir  / "hard_hat_workers3179.png"
image_conversion(test_image , "hard_hat_workers3179.jpg")

display.display(
    Image.open(test_image) ,
    Image.open("hard_hat_workers3179.jpg")
)
```



For trianing the yolo format is :

data_yolo --images -----train -----val --labels -----train -----val

```
yolo_base = Path("yolo_data")

# clear the previous
shutil.rmtree(yolo_base , ignore_errors=True)

(yolo_base / "images" / "train").mkdir(parents=True, exist_ok=True)
(yolo_base / "images" / "val").mkdir(parents=True ,exist_ok=True)
(yolo_base / "labels" / "train").mkdir(parents=True , exist_ok=True)
(yolo_base / "labels" / "val").mkdir(parents=True , exist_ok=True)

!tree $yolo_base
```

```
yolo_data
├── images
│   ├── train
│   └── val
└── labels
    ├── train
    └── val

6 directories, 0 files
```

```
print(annotations_dir)
print(images_dir)
print("/kaggle/working/data_images/annotations")
```

```
/kaggle/working/data_images/annotations
/kaggle/working/data_images/images
/kaggle/working/data_images/annotations
```

```
len(list(annotations_dir.iterdir()))
```

```
5000
```

```
Start coding or generate with AI.
```

```
train_frac = 0.8
images = list(images_dir.glob("*"))

for img in tqdm(images):
    split = "train" if random.random() < train_frac else "val"

    annotation = annotations_dir / f"{img.stem}.xml"
    # This might raise an error:
    try:
        parsed = parse_annotations(annotation)
    except Exception as e:
        print(f"Faild to Prase {img.stem}. Skipping.")
        print(e)
        continue

    dest = yolo_base / "labels" / split / f"{img.stem}.txt"
    write_label(parsed, dest)

    dest = yolo_base / "images" / split / f"{img.stem}.jpg"
    image_conversion(img, dest)
```

```
  0%|          | 0/5000 [00:00<?, ?it/s]
```

```
train_count = len(list((yolo_base / "images" / "train").glob("*")))
val_count = len(list((yolo_base / "images" / "val").glob("*")))
total_count = train_count + val_count

print(f"Training fraction:   {train_count/total_count:0.3f}")
print(f"Validation fraction: {val_count/total_count:0.3f}")
```

```
Training fraction:   0.797
Validation fraction: 0.203
```

```
# yaml is the file that we give when training the model yolo ,
# it also kind of dictionary
metadata = {
    "path" : str(yolo_base.absolute()),
    "train": "images/train" ,
    "val"  :  "images/val" ,
    "names" : classes ,
    "nc"  : len(classes)
}
print(metadata)
```

```
{'path': '/kaggle/working/yolo_data', 'train': 'images/train', 'val': 'images/val', 'names': ['helmet', 'head', 'pe
```

```
yolo_config = "data.yaml"
yaml.safe_dump(metadata , open(yolo_config , "w"))

!cat data.yaml
```

```
names:
- helmet
- head
- person
nc: 3
path: /kaggle/working/yolo_data
train: images/train
val: images/val
```

```
# now we are gonna import the model
# we import the yolo nano model
```

```
model = YOLO("yolov8n.pt")
print(model)
```

Show hidden output

```
len(model.names)# number of objects yolo can detect , coco dataset
```

```
80
```

```
model.train(
    data="data.yaml",       # ✅ Your dataset config file
    epochs=10,              # 🔁 Number of training epochs
    imgsz=640,              # 🖼️ Image size (usually 640 or 512)
    batch=16,               # 🧠 Batch size (reduce if OOM error)
    name="helmet-detect",   # 🏷️ Folder name in runs/detect/
    project="runs/detect",  # 📁 Where to store training logs & weights
    device=0,               # 🖥️ 0 = GPU, 'cpu' if no GPU
    pretrained=True,        # ✅ Use pretrained YOLOv8 weights
    workers=2,              # 🧵 Dataloader workers (lower for Kaggle)
    lr0=0.01,               # 📈 Learning rate (optional tweak)
)
```

Show hidden output

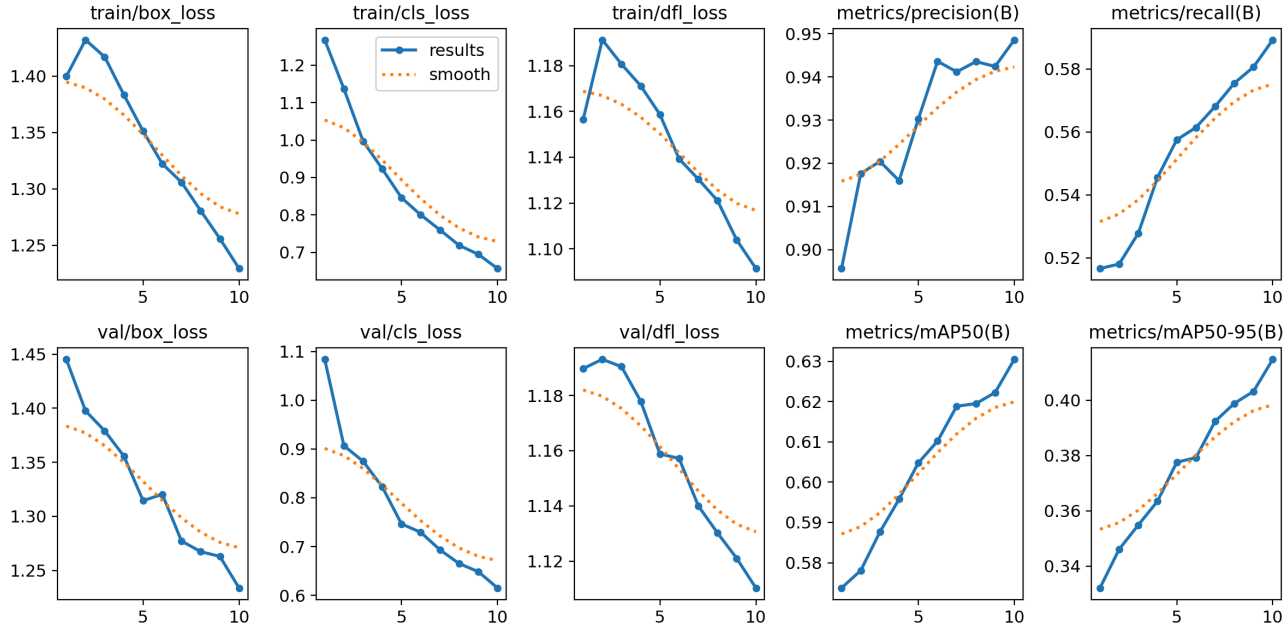
```
# we saw the results of this ,
save_dir =  Path("/kaggle/working/runs/detect/helmet-detect3")
```

```
!tree $save_dir
```

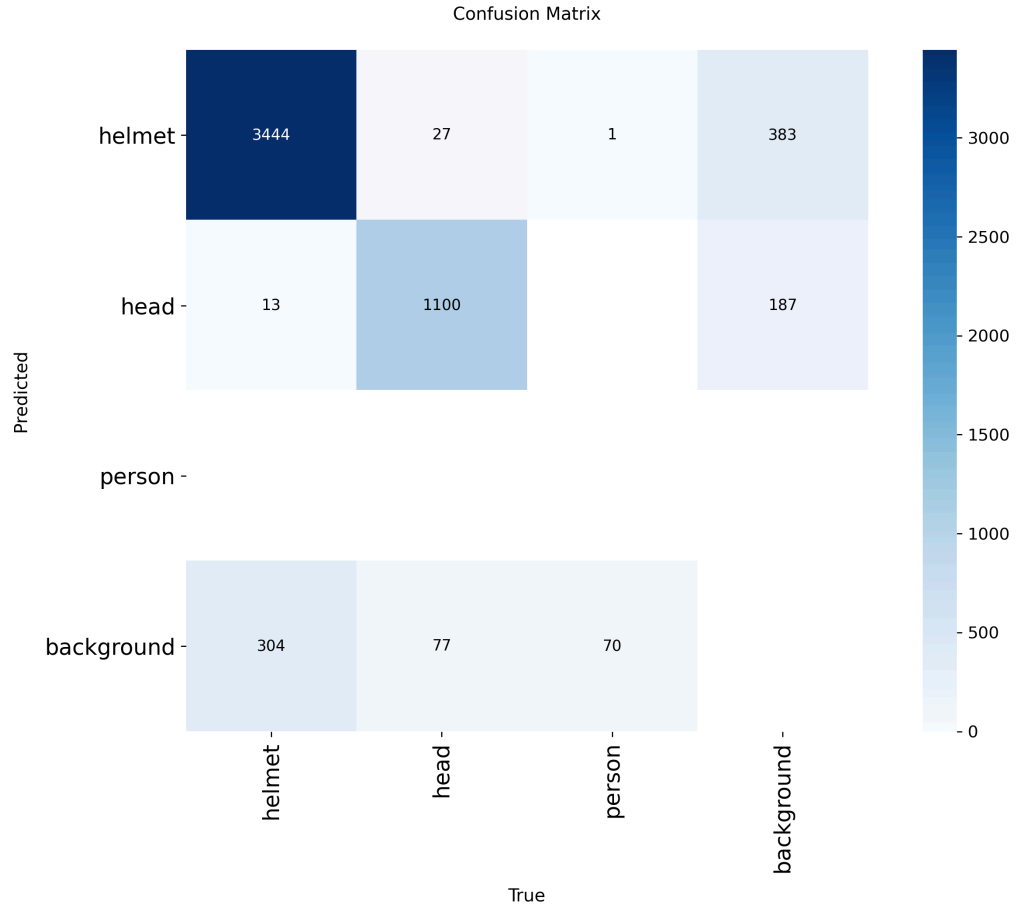
```
/kaggle/working/runs/detect/helmet-detect3
├── args.yaml
├── BoxF1_curve.png
├── BoxP_curve.png
├── BoxPR_curve.png
├── BoxR_curve.png
├── confusion_matrix_normalized.png
├── confusion_matrix.png
├── labels_correlogram.jpg
├── labels.jpg
├── results.csv
├── results.png
├── train_batch0.jpg
├── train_batch1.jpg
├── train_batch2.jpg
├── val_batch0_labels.jpg
├── val_batch0_pred.jpg
├── val_batch1_labels.jpg
├── val_batch1_pred.jpg
├── val_batch2_labels.jpg
├── val_batch2_pred.jpg
└── weights
    ├── best.pt
    └── last.pt

1 directory, 22 files
```

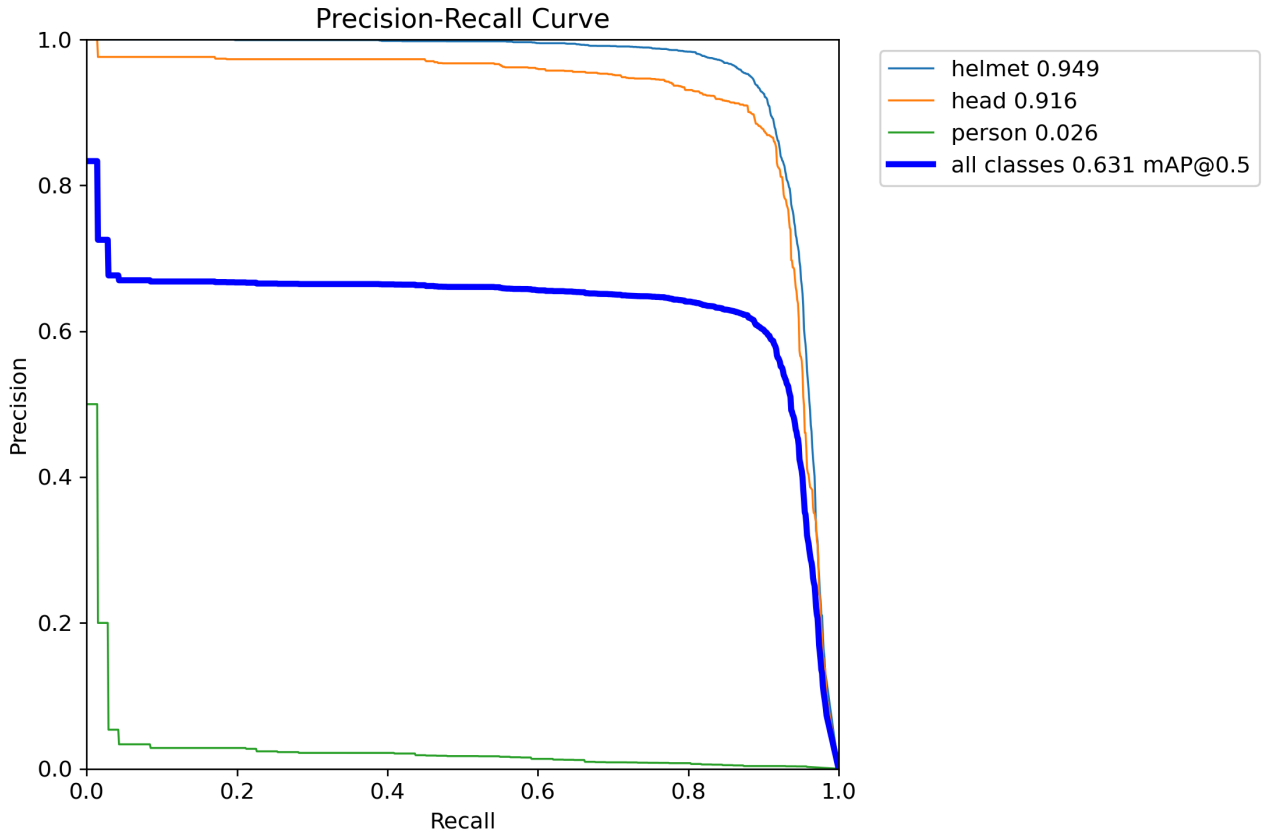
```
#Summary of loss, precision, recall, mAP
#Check if your model is learning and improving
pr_curve_path = save_dir / "results.png"
Image.open(pr_curve_path)
```




```
#✅ Prediction quality per class
#See what classes are confused
cf_dir= save_dir / "confusion_matrix.png"
Image.open(cf_dir)
```



```
#Precision vs Recall curve
#Measure class-wise balance between precision & recall
BoxPR_dir= save_dir / "BoxPR_curve.png"
Image.open(BoxPR_dir)
```

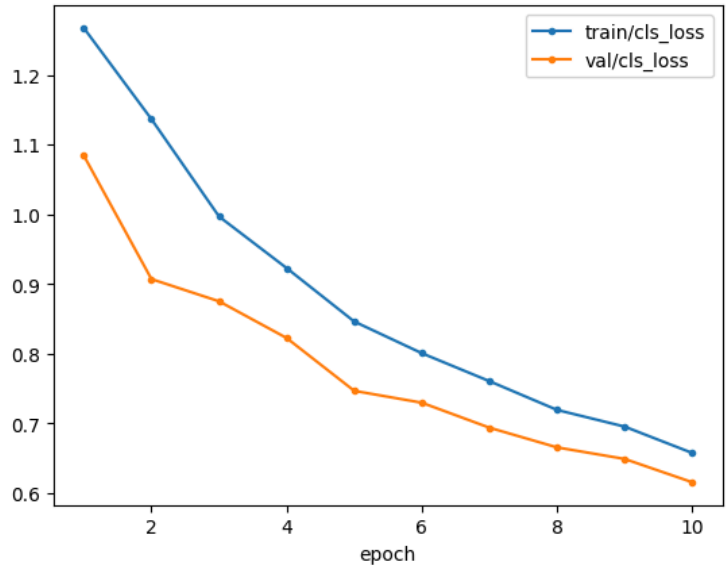



```
#Per-epoch metrics (mAP, loss, etc.)
#Analyze model performance over time
df = pd.read_csv(save_dir / "results.csv", skipinitialspace=True).set_index(
    "epoch"
)
df.head()
```

epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP5
1	48.8394	1.39971	1.26732	1.15649	0.89562	0.51661	0.5
2	94.7687	1.43218	1.13645	1.19130	0.91763	0.51803	0.5
3	140.2980	1.41703	0.99669	1.18067	0.92041	0.52784	0.5
4	185.2830	1.38343	0.92261	1.17121	0.91596	0.54558	0.5
5	230.4810	1.35123	0.84587	1.15868	0.93024	0.55751	0.6

```
df[['train/cls_loss' , 'val/cls_loss']].plot(marker='.')
```

```
<Axes: xlabel='epoch'>
```




```
#Validation predictions (visual)
#Visually inspect how well your model detects
val_batch_dir= save_dir / "val_batch2_pred.jpg"
Image.open(val_batch_dir)
```


```
model = YOLO("/kaggle/working/runs/detect/helmet-detect3/weights/best.pt")
model.train(
    data="data.yaml",       # ✅ Your dataset config file
    epochs=10,              # 🔁 Number of training epochs
    imgsz=640,              # 🖼️ Image size (usually 640 or 512)
    batch=16,               # 🧠 Batch size (reduce if OOM error)
    name="helmet-detect",   # 🏷️ Folder name in runs/detect/
    project="runs/detect",  # 📁 Where to store training logs & weights
    device=0,               # 🖥️ 0 = GPU, 'cpu' if no GPU
    pretrained=True,        # ✅ Use pretrained YOLOv8 weights
    workers=2,              # 🧵 Dataloader workers (lower for Kaggle)
    lr0=0.01,               # 📈 Learning rate (optional tweak)
)
```

Show hidden output

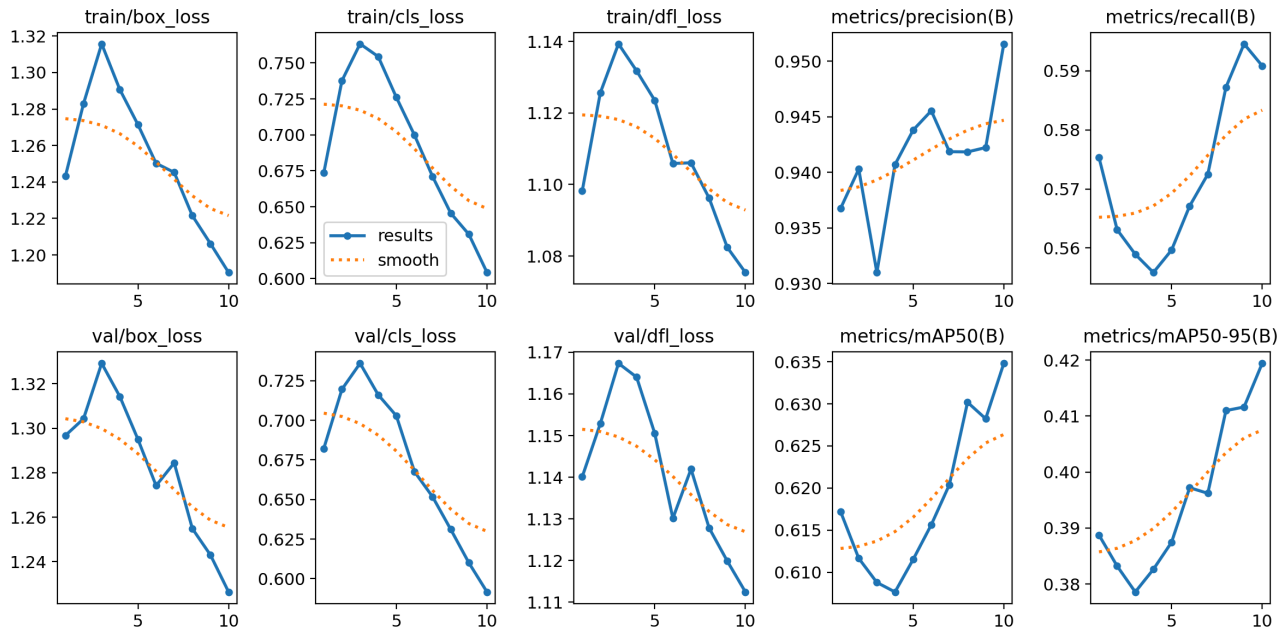
```
save_dir1 = Path("/kaggle/working/runs/detect/helmet-detect4")
```

```
!tree $save_dir1
```

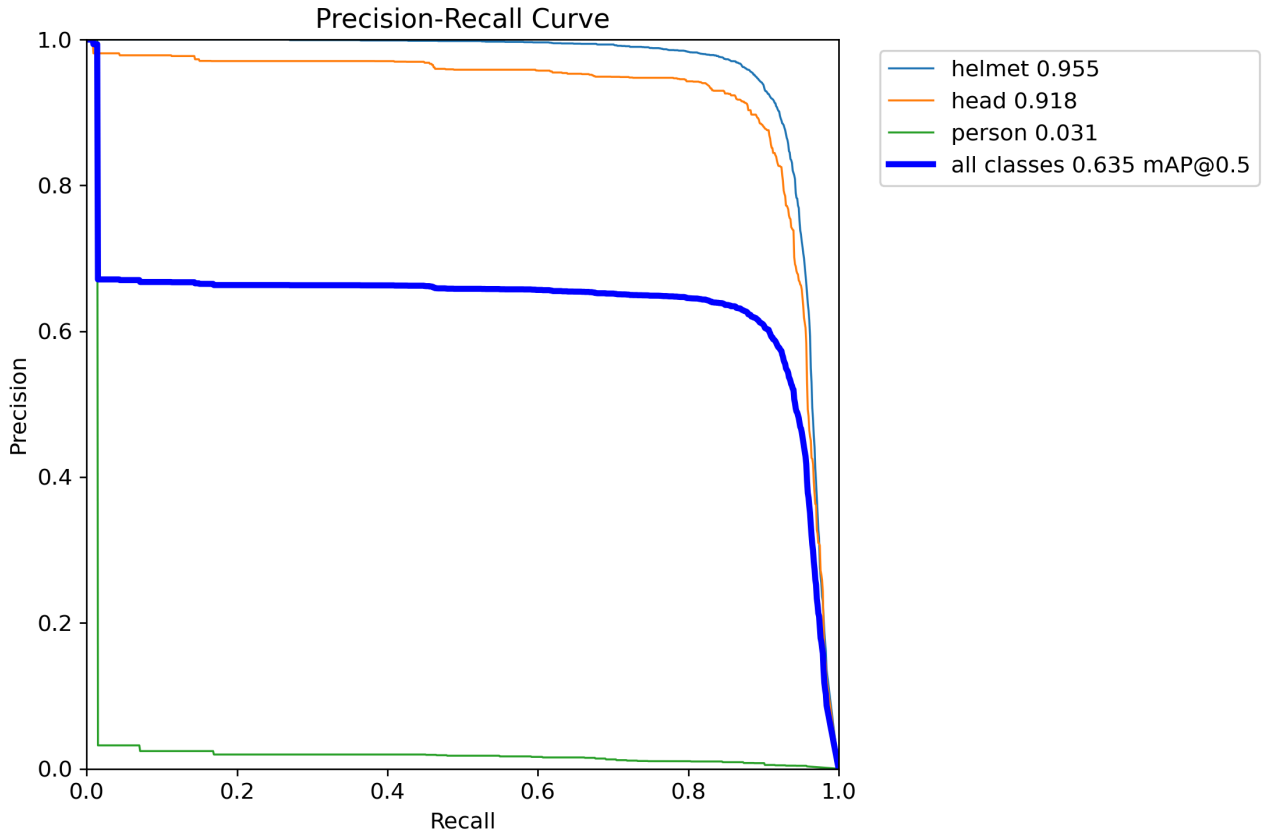
```
/kaggle/working/runs/detect/helmet-detect4
├── args.yaml
├── BoxF1_curve.png
├── BoxP_curve.png
├── BoxPR_curve.png
├── BoxR_curve.png
├── confusion_matrix_normalized.png
├── confusion_matrix.png
├── labels_correlogram.jpg
├── labels.jpg
├── results.csv
├── results.png
├── train_batch0.jpg
├── train_batch1.jpg
├── train_batch2.jpg
├── val_batch0_labels.jpg
├── val_batch0_pred.jpg
├── val_batch1_labels.jpg
├── val_batch1_pred.jpg
├── val_batch2_labels.jpg
├── val_batch2_pred.jpg
└── weights
    ├── best.pt
    └── last.pt

1 directory, 22 files
```

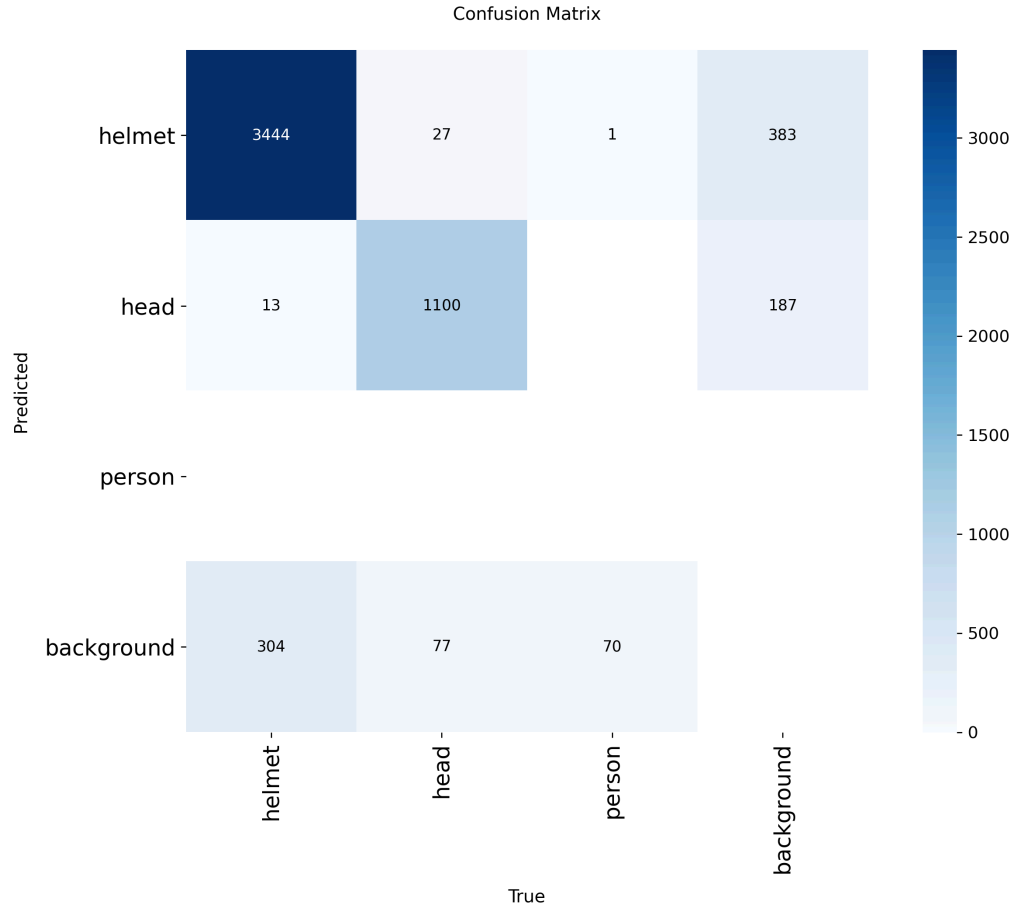
```
pr_curve_path = save_dir1 / "results.png"
Image.open(pr_curve_path)
```




```
BoxPR_dir= save_dir1 / "BoxPR_curve.png"
Image.open(BoxPR_dir)
```




```
f_dir= save_dir1 / "confusion_matrix.png"
Image.open(cf_dir)
```

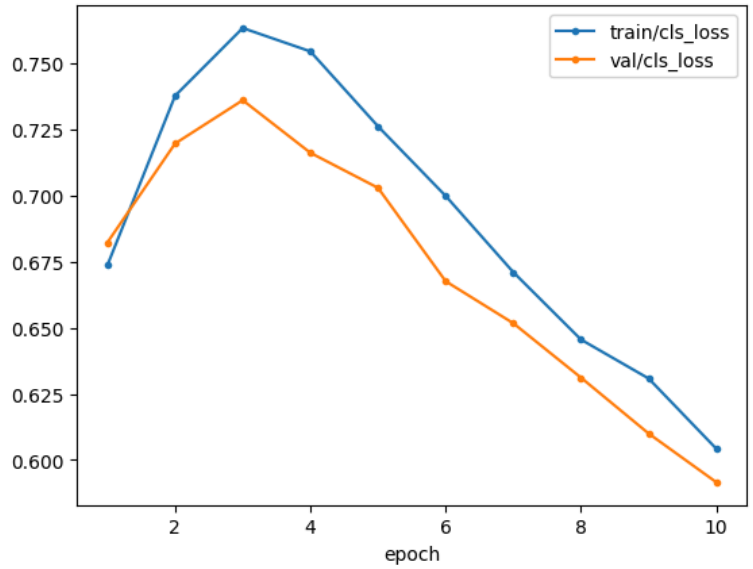



```
df = pd.read_csv(save_dir1 / "results.csv", skipinitialspace=True).set_index(
    "epoch"
)
df.head()
```

epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP5
1	48.2050	1.24314	0.67361	1.09817	0.93677	0.57532	0.6
2	95.7471	1.28298	0.73763	1.12567	0.94030	0.56310	0.6
3	144.1220	1.31573	0.76324	1.13934	0.93100	0.55894	0.6
4	192.0050	1.29069	0.75441	1.13179	0.94071	0.55585	0.6
5	240.6170	1.27143	0.72608	1.12354	0.94381	0.55964	0.6

```
df[['train/cls_loss' , 'val/cls_loss']].plot(marker='.')
```

```
<Axes: xlabel='epoch'>
```




```
model = YOLO("/kaggle/working/runs/detect/helmet-detect4/weights/best.pt")
model.train(
    data="data.yaml",       # ✅ Your dataset config file
    epochs=20,              # 🔁 Number of training epochs
    imgsz=640,              # 🖼️ Image size (usually 640 or 512)
    batch=16,               # 🧠 Batch size (reduce if OOM error)
    name="helmet-detect",   # 🏷️ Folder name in runs/detect/
    project="runs/detect",  # 📁 Where to store training logs & weights
    device=0,               # 🖥️ 0 = GPU, 'cpu' if no GPU
    workers=2,              # 🧵 Dataloader workers (lower for Kaggle)
    lr0=0.001,               # 📈 Learning rate (optional tweak)
)
```

Show hidden output

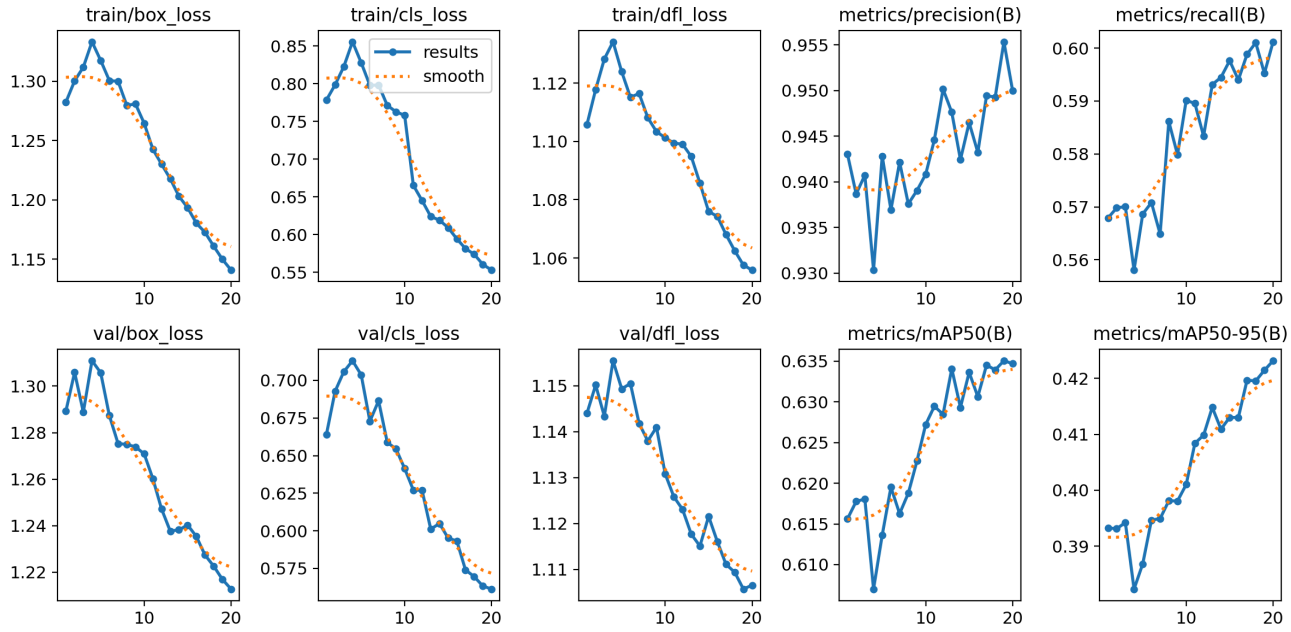
```
save_dir2 = Path("/kaggle/working/runs/detect/helmet-detect5")
```

```
!tree $save_dir2
```

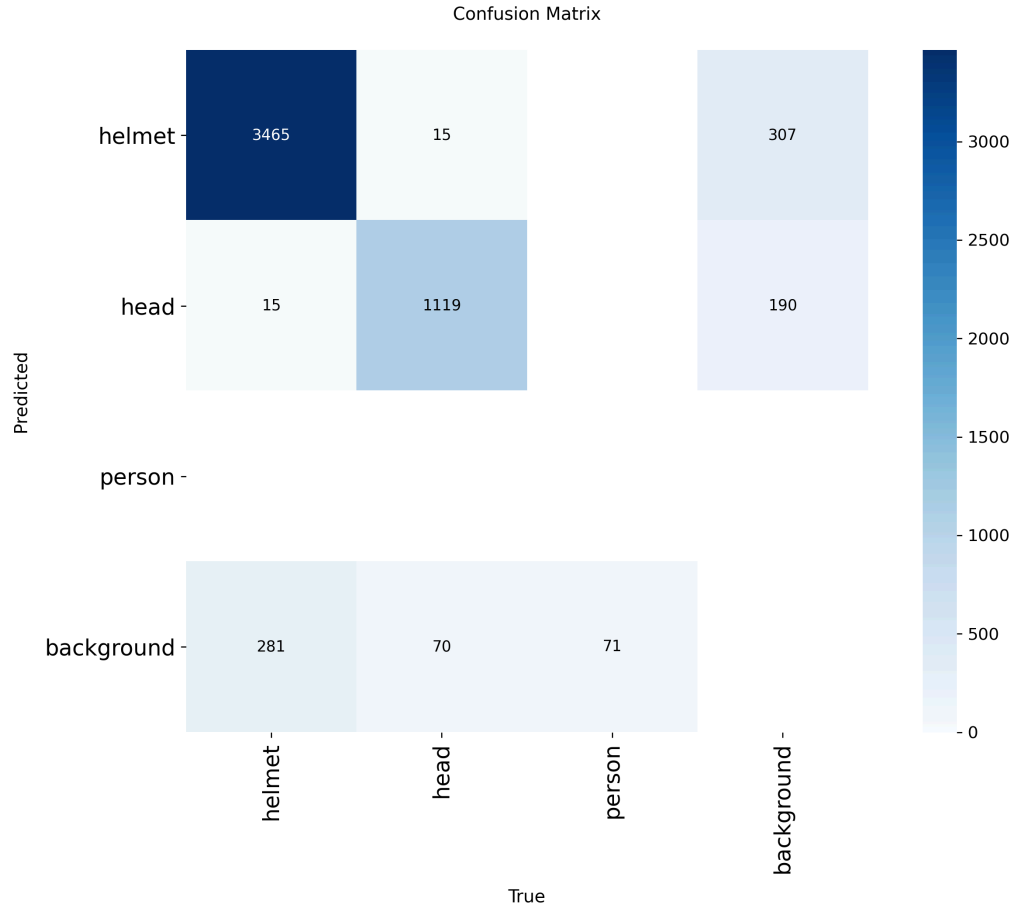
```
/kaggle/working/runs/detect/helmet-detect5
├── args.yaml
├── BoxF1_curve.png
├── BoxP_curve.png
├── BoxPR_curve.png
├── BoxR_curve.png
├── confusion_matrix_normalized.png
├── confusion_matrix.png
├── labels_correlogram.jpg
├── labels.jpg
├── results.csv
├── results.png
├── train_batch0.jpg
├── train_batch1.jpg
├── train_batch2490.jpg
├── train_batch2491.jpg
├── train_batch2492.jpg
├── train_batch2.jpg
├── val_batch0_labels.jpg
├── val_batch0_pred.jpg
├── val_batch1_labels.jpg
├── val_batch1_pred.jpg
├── val_batch2_labels.jpg
├── val_batch2_pred.jpg
└── weights
    ├── best.pt
    └── last.pt

1 directory, 25 files
```

```
pr_curve_path = save_dir2 / "results.png"
Image.open(pr_curve_path)
```



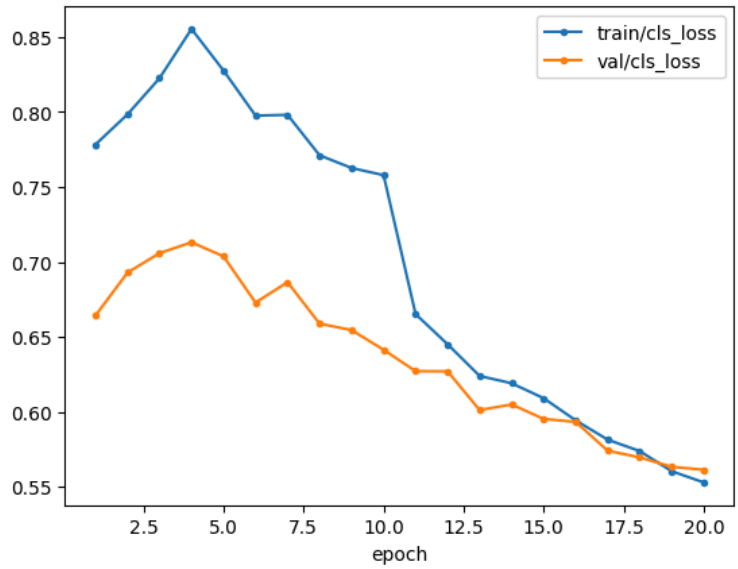

```
result_path = save_dir2 / "confusion_matrix.png"
Image.open(result_path)
```



```
df = pd.read_csv(save_dir2 / "results.csv", skipinitialspace=True).set_index(
    "epoch"
)

df[['train/cls_loss' , 'val/cls_loss']].plot(marker='.')
```

```
<Axes: xlabel='epoch'>
```




```
def model_results(save_path):
    pr_curve_path = save_path / "results.png"
    Image.open(pr_curve_path)
```